AEE-331
HEAT TRANSFER
Project Assignment
Project Question 2

OZAN ÖNDER-22110151003
MEHMET AKIN-21110111012
İBRAHİM HAKKI KOÇ-22110111023
OSMAN BİLER-22110111037

December 22, 2024

Contributions

Mehmet and Ozan derived the necessary formulas and wrote the code for the numerical method in MATLAB. Ibrahim created the code for the analytical method and, in the end, compared the two plots. However, overall, it was a collective effort; we handled the methodology, analysis, and report together as a team.

Contents

1 Introduction **3**

2 Methodology and Results **3**

3 Conclusion **10**

4 Appendix **11**

4.1 Code of Numerical Method 11

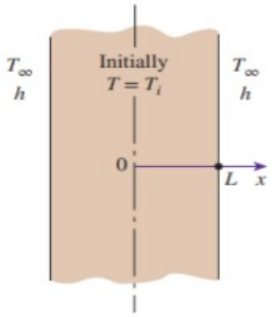
4.2 Code of Analytical Method 12

4.3 Code of Comparison Numerical and Analytical Methods 14

1 Introduction

In this study, we are solving 1-D transient conduction in a plane wall subjected to convection from all surfaces.

We can see the question in Figure 1.



A large plane wall

Consider 1-D, transient, constant properties problem.

- $2L = 0.2\ m$
- $k_{wall} = 20\frac{W}{m°C}, c_p = 500\frac{j}{kg°C}, \rho = 1000\ kg/m^2$
- $T_i = 100\ °C, T_\infty = 20\ °C, h = 60\frac{W}{m^2°C}$

a) Derive formulations via energy balance method. You may choose explicit or implicit approach.

b) Solve the equations with a suitable method and obtain the time-dependent temperature distribution. Use a minimum of 10 grids for L. Move forward in time until the solution reaches steady state.

c) Plot the temperature distribution at 15 second intervals (0, 15, 30,…).

d) Obtain the temperatures using the analytical solution in the textbook for every 15-second time intervals (**Do not use one-term solution unless applicable: $\tau > 0.2$**):
To draw the analytical solution, it may be sufficient to calculate temperatures at 5 midpoints and connect them (with boundary temperatures) with a spline.

Compare numerical and analytical solutions in the same plot.

Figure 1. Project Question

2 Methodology and Results

First of all, we need to derive formulations via energy balance method.

The general energy balance equation is:

$$\sum_{\text{All sides}} \dot{Q}^i + \dot{E}^i_{\text{gen, element}} = \rho V_{\text{element}} c_p \frac{T_m^{i+1} - T_m^i}{\Delta t}$$

We choose the explicit approach and derive formulations for center, right boundary and way points. Also, in this question there is no heat generation.

a)

For center (x=0) derived with symmetry condition;

$$T_m^{i+1} = T_m^i + 2\frac{\alpha\,\Delta t}{(\Delta x)^2}\left[T_{m+1}^i - T_m^i\right]$$

For right boundary (x=L);

$$T_m^{i+1} = T_m^i + \frac{2\alpha\Delta t}{(\Delta x)^2}\,(T_{m-1}^i - T_m^i) + \frac{2h\Delta t}{\rho c_p \Delta x}(T_\infty - T_m^i)$$

For way points;

$$T_m^{i+1} = \tau(T_{m-1}^i + T_{m+1}^i) + (1-2\tau)T_m^i$$

b-c)

We entered MATLAB to question parameters firstly.

```
L = 0.1;
k_wall = 20;
rho = 1000;
c_p = 500;
h = 60;
T_i = 100;
T_inf = 20;
alpha = k_wall / (rho * c_p);
Bi = h * L / k_wall;
```

Figure 2. Parameters

We choose 10 grids with including center and right boundary. Also we check the stability condition for explicit method:

```
n = 10;
dx = L / (n-1);
Fo = alpha * 0.5 / (dx^2); % Fourier Number (dt=0.5 taken constant)
if Fo > 0.5
    error('The Fourier number does not ensure stability; reduce dt or dx.');
end
```

After that we created a matrix for temperature values and entered the formulas we derived before:

```
T = T_i * ones(n, 1); % Initial temperature profile
T_new = T;
time = 0;
numeric_results = [];

while abs(T(1) - T_inf) > tol
    if mod(time, time_step_interval) == 0
        numeric_results = [numeric_results, T];
    end

    for i = 2:(n-1)
        T_new(i) = Fo * (T(i+1) + T(i-1)) + (1 - 2 * Fo) * T(i);
    end

    % x=L (right boundary condition)
    T_new(end) = T(end) + 2 * alpha * dt / dx^2 * (T(end-1) - T(end)) + 2 * h * dt / (rho * c_p * dx) * (T_inf - T(end)

    % x=0 (symmetry)
    T_new(1) = T(1) + 2 * alpha * dt / dx^2 * (T(2) - T(1));

    T = T_new;
    time = time + dt;
end
```

When we calculated the temperatures, we plotted the temperature distribution.

```
figure;
hold on;

% Numerical results
selected_times = 0:30:(max_iterations-1)*dt_analytical; % Adjust to show 0, 30, 60, ... seconds
selected_indices = ismember(0:time_step_interval:(max_iterations-1)*time_step_interval, selected_times);

for i = find(selected_indices)
    plot(x_numeric, numeric_results(:, i), '-', 'DisplayName', sprintf('Numerical t = %d s', (i-1)*dt_analytical));
end
```

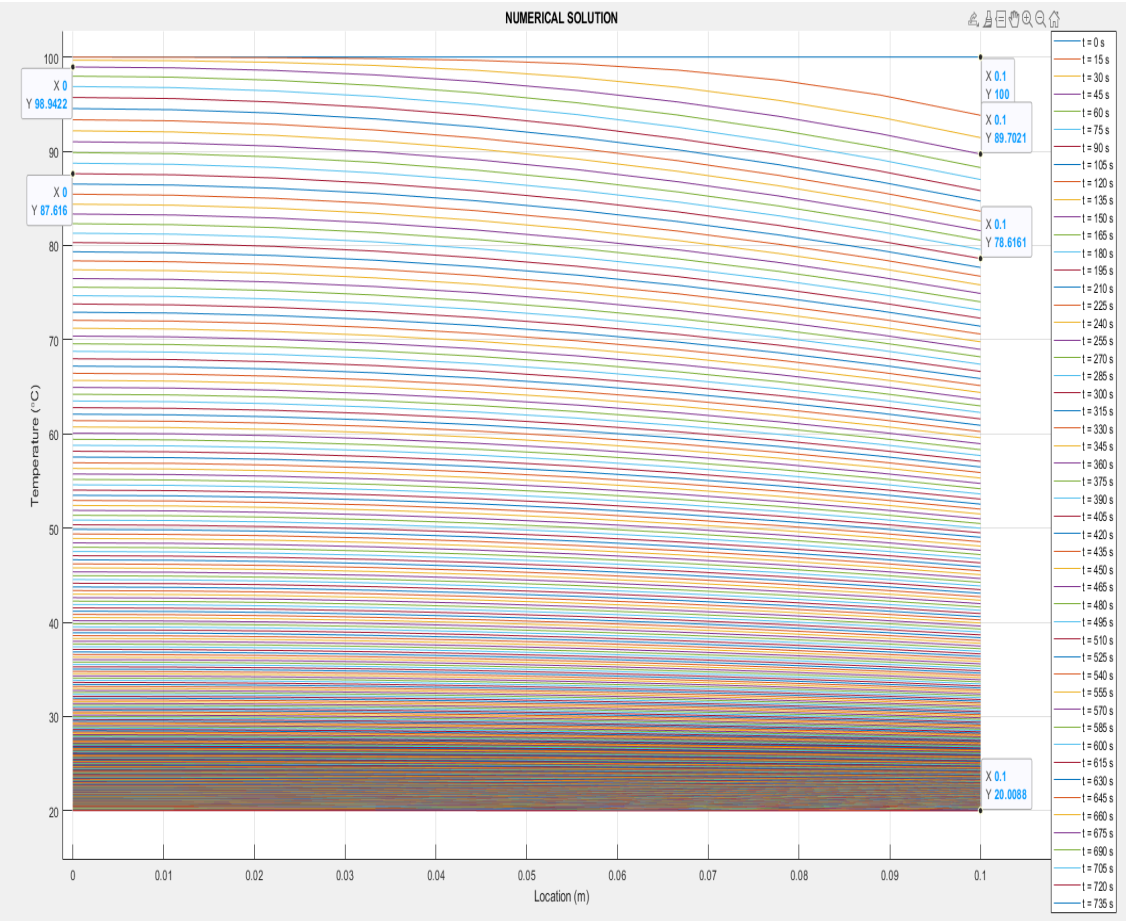


Figure 3. Temperature Distribution with Numerical Solution

d) On the option d we need to do an analytical solution. We need lamda values to solve in analytical method. Since we couldn't find the lambda values using MATLAB, we assigned the lambda values ourselves to our code to find temperatures at 5 midpoints:

```
% Given lambda values
lambda = [0.521791176313584, 3.23408975864280, 6.33053920823218, 9.45649156363962, 12.5901941739888];

% Spatial steps
dx = 0.025;         % Spatial step (m) for 5 points dx = 0.025
x = 0:dx:L;         % Spatial range

% Initial time and tolerance
t = 15;             % Initial time step 15 seconds
tolerance = 0.01;   % Steady-state tolerance
steady_state_reached = false;

% Solution matrix, initially only one time step (t = 0)
T_solution = T_i * ones(length(x), 1); % Initially, 100°C at every point

% Time step counter
t_idx = 1;  % Initial time step counter
```

We determined the Fourier number because if the Fourier number is greater than 0.2, we used the term approximation method; otherwise, we used the exact solution method. The formulation of exact solution is;

$$\theta = \sum_{n=1}^{\infty} \frac{4 \sin \lambda_n}{2\lambda_n + \sin(2\lambda_n)} e^{-\lambda_n^2 \tau} \cos\left(\lambda_n \frac{x}{L}\right)$$

$$\lambda_n \tan(\lambda_n) = \mathrm{Bi}$$

When fo>0.2 , we use one-term approximation.

$$T_{\text{solution}}(i, t_{\text{idx}} + 1) = \left(\frac{4\sin(\lambda_1)}{2\lambda_1 + \sin(2\lambda_1)}\right) \cdot e^{-\lambda_n^1 \cdot \mathrm{Fo}} \cdot \cos\left(\frac{\lambda_1 \cdot x(i)}{L}\right) \cdot (T_i - T_\infty) + T_\infty$$

Subsequently, the temperature approaches 20°C. If the maximum temperature difference is smaller than the tolerance, the system reaches a steady state.

```
while ~steady_state_reached
    Fo = alpha * t / L^2;

    T_previous = T_solution(:, t_idx);

    if Fo < 0.2
        % Exact solution (5 terms of the infinite series)
        for i = 1:length(x)
            theta_sum = 0;
            for n = 1:length(lambda)
                theta_n = (4 * sin(lambda(n))) / (2 * lambda(n) + sin(2 * lambda(n))) ...
                          * exp(-lambda(n)^2 * Fo) * cos(lambda(n) * x(i) / L);
                theta_sum = theta_sum + theta_n;
            end
            T_solution(i, t_idx + 1) = theta_sum * (T_i - T_inf) + T_inf; % Actual temperature
        end
    else
        % One-term solution
        n = 1; % we use just first term
        for i = 1:length(x)
            T_solution(i, t_idx + 1) = (4 * sin(lambda(n))) / (2 * lambda(n) + sin(2 * lambda(n))) ...
                * exp(-lambda(n)^2 * Fo) * cos(lambda(n) * x(i) / L) * (T_i - T_inf) + T_inf;
        end
    end

    % Steady-state check: If the temperature change between consecutive time steps is very small, steady state is reached
    max_temp_change = max(abs(T_solution(:, t_idx + 1) - T_previous)); % Calculate the maximum temperature change

    if max_temp_change < tolerance
        steady_state_reached = true; % Steady state is reached
    else
        % Increment time by 15 seconds
        t = t + 15;
        t_idx = t_idx + 1; % Move to the next time step
    end
end

% Plot the graph
figure;
plot(x, T_solution(:, 1:t_idx), line, 2);
xlabel('Position x (m)');
ylabel('Temperature T (°C)');
title('Exact and One-Term Approximation - Steady State Achievement');
legend(arrayfun(@(t) ['t = ' num2str((t-1) * 15) ' s'], 1:t_idx, 'UniformOutput', false));
grid on;
```

Later, we plotted the temperature distribution of analytical solution:

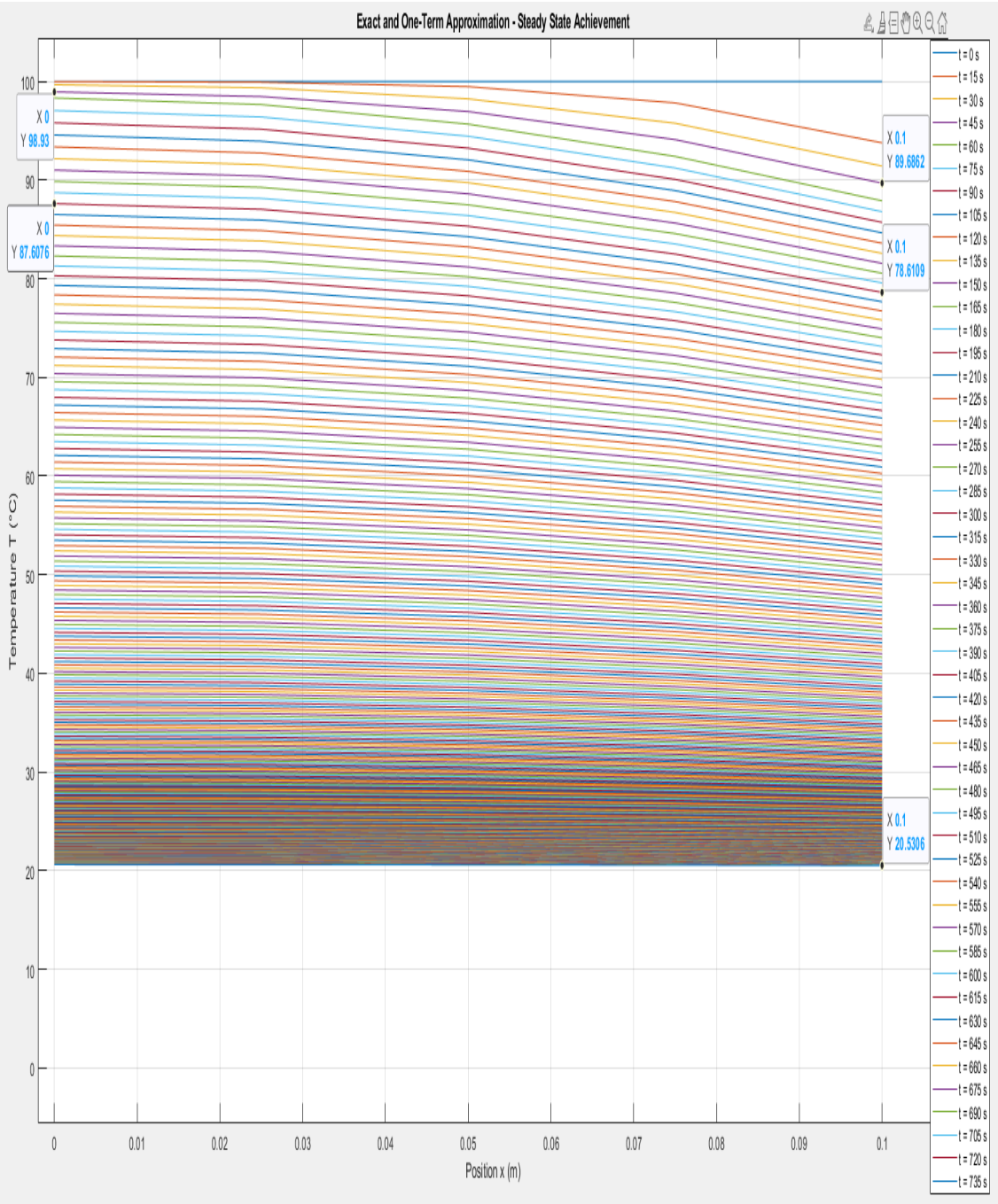


Figure 4. Temperature Distribution with Analytical Solution

Finally, we plotted numerical and analytical solution in the same figure to compare them:

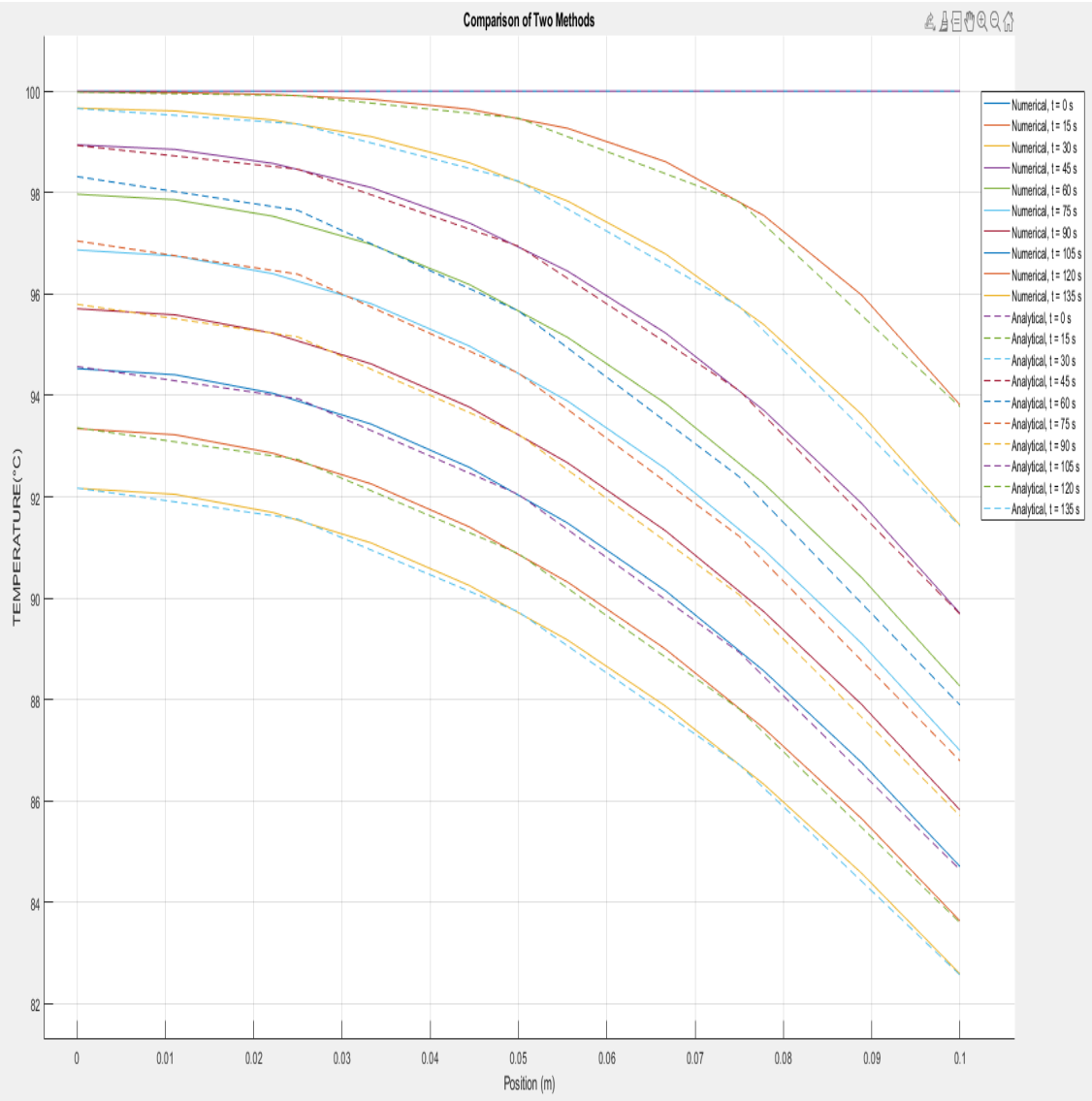


Figure 5. Comparison of Numerical and Analytical Graphs

3 Conclusion

In this study, we solved the 1-D transient heat conduction problem with numerical and analytical methods. The results of both approaches supported each other, demonstrating the accuracy of our methods. The temperature distributions we obtained reached equilibrium values in time, in line with theoretical expectations.

4 Appendix

4.1 Code of Numerical Method

```
L = 0.1;    % Length of the rod
k = 20;     % Thermal conductivity
cp = 500;   % Specific heat capacity
rho = 1000; % Density
h = 60;     % Convection coefficient
T_inf = 20; % Surrounding temperature
T_initial = 100; % Initial temperature of the rod
n = 10;     % Number of points
dx = L / (n-1);  % Adjust dx to ensure 10 points including boundaries
alpha = k / (rho * cp); % Thermal diffusivity
dt = 0.5;   % Time step
Fo = alpha * dt / (dx^2); % Fourier number

tol = 0.01; % Tolerance value

if Fo > 0.5
error('The Fourier number does not ensure stability; either dt or dx should be reduced.');
end

T = T_initial * ones(n, 1); % Adjust T array to have n points
T_new = T;

time = 0;
results = [];

while abs(T(1) - T_inf) > tol
    if mod(time, 15) == 0
        results = [results, T];
    end

    for i = 2:(n-1)
        T_new(i) = Fo * (T(i+1) + T(i-1)) + (1 - 2 * Fo) * T(i);
    end

    % x=L (right boundary condition)
    T_new(end) = T(end) + 2 * alpha * dt / dx^2 * (T(end-1) - T(end)) + ...
                 2 * h * dt / (rho * cp * dx) * (T_inf - T(end));

    % x=0 (symmetry)
    T_new(1) = T(1) + 2 * alpha * dt / dx^2 * (T(2) - T(1));

    T = T_new;
```

```
    time = time + dt;
end

x = linspace(0, L, n); % Adjust x to have n points
figure;
hold on;
for i = 1:size(results, 2)
    plot(x, results(:, i), 'DisplayName', ['t = ', num2str((i-1)*15), ' s']);
end
legend('show');
xlabel('Location (m)');
ylabel('Temperature (\circC)');
title('Temperature Distribution');
grid on;
```

4.2 Code of Analytical Method

```
% Parameters
L = 0.1;            % Half wall thickness (m)
k_wall = 20;        % Thermal conductivity (W/mK)
h = 60;             % Convection coefficient (W/m^2K)
T_i = 100;          % Initial temperature ( C )
T_inf = 20;         % Ambient temperature ( C )
alpha = k_wall / (1000 * 500); % Thermal diffusivity (m^2/s)

% Given lambda roots
lambda = [0.521791176313584, 3.23408975864280, 6.33053920823218, 9.45649156363962, 12.5901941739888];

% Spatial steps
dx = 0.025;          % Spatial step (m); for 5 points, dx = 0.025
x = 0:dx:L;          % Spatial domain

% Initial time and tolerance
t = 15;              % Initial time step, 15 seconds
tolerance = 0.01;    % Steady-state tolerance
steady_state_reached = false;

% Solution matrix; initially contains only the first time step (t = 0)
T_solution = T_i * ones(length(x), 1); % Initial temperature of 100 C  everywhere

% Time step counter
t_idx = 1;  % Initial time step counter

% Solution calculation
while ~steady_state_reached
```

```
    % Compute Fourier number (Fo) at each time step
    Fo = alpha * t / L^2;  % Fourier number

    T_previous = T_solution(:, t_idx); % Temperatures at the previous time step

    if Fo < 0.2
        % Exact solution (5 terms of the infinite series)
        for i = 1:length(x)
            theta_sum = 0;
            for n = 1:length(lambda)
                theta_n = (4 * sin(lambda(n))) / (2 * lambda(n) + sin(2 * lambda(n))) ...
                        * exp(-lambda(n)^2 * Fo) * cos(lambda(n) * x(i) / L);
                theta_sum = theta_sum + theta_n;
            end
            T_solution(i, t_idx + 1) = theta_sum * (T_i - T_inf) + T_inf; % Actual temperature
        end
    else
        % One-term solution
        n = 1; % First term
        for i = 1:length(x)
            T_solution(i, t_idx + 1) = (4 * sin(lambda(n))) / (2 * lambda(n) + sin(2 * lambda(n))) ...
                * exp(-lambda(n)^2 * Fo) * cos(lambda(n) * x(i) / L) * (T_i - T_inf) + T_inf;
        end
    end

    % Steady-state check: If temperature change between consecutive time steps is very small, steady state is reached
    max_temp_change = max(abs(T_solution(:, t_idx + 1) - T_previous)); % Maximum temperature change

    if max_temp_change < tolerance
        steady_state_reached = true; % Steady state is achieved
    else
        % Increase time by 15 seconds
        t = t + 15;
        t_idx = t_idx + 1; % Move to the next time step
    end
end

% Plotting
figure;
plot(x, T_solution(:, 1:t_idx), 'LineWidth', 2);
xlabel('Position x (m)');
```

```
ylabel('Temperature T ( C )');
title('Exact and One-Term Approximation - Steady State Achievement');
legend(arrayfun(@(t) ['t = ' num2str((t-1) * 15) ' s'], 1:t_idx, 'UniformOutput', false));
grid on;
```

4.3 Code of Comparison Numerical and Analytical Methods

```
% Common parameters
L = 0.1;            % Half wall thickness (m)
k = 20;             % Thermal conductivity (W/mK)
h = 60;             % Convection coefficient (W/m^2K)
T_inf = 20;         % Ambient temperature ( C )
T_initial = 100;    % Initial temperature ( C )
rho = 1000;         % Density (kg/m^3)
cp = 500;           % Specific heat (J/kgK)
alpha = k / (rho * cp); % Thermal diffusivity (m^2/s)

% Parameters (Numerical Method)
n = 10;           % Number of spatial points
dx = L / (n-1);   % Spatial step (m)
dt = 0.5;         % Time step (s)
Fo = alpha * dt / (dx^2);

if Fo > 0.5
    error('The Fourier number does not ensure stability; either dt or dx should be reduced.');
end

T1 = T_initial * ones(n, 1); % Initial temperatures for numerical method
T1_new = T1;

time1 = 0;
results1 = [];

while abs(T1(1) - T_inf) > 1e-3 && time1 <= 135
    if mod(time1, 15) == 0
        results1 = [results1, T1];
    end

    for i = 2:(n-1)
        T1_new(i) = Fo * (T1(i+1) + T1(i-1)) + (1 - 2 * Fo) * T1(i);
    end

    % x=L (right boundary condition)
```

```
    T1_new(end) = T1(end) + 2 * alpha * dt / dx^2 * (T1(end-1) - T1(end)) + 2 * h * dt / (rho * cp * dx) * (T_inf - T1(end));

    % x=0 (symmetry)
    T1_new(1) = T1(1) + 2 * alpha * dt / dx^2 * (T1(2) - T1(1));

    T1 = T1_new;
    time1 = time1 + dt;
end

x1 = linspace(0, L, n);

% Parameters (Analytical Method)
lambda = [0.521791176313584, 3.23408975864280, 6.33053920823218, 9.45649156363962, 12.5901941739888];
dx2 = 0.025; % Spatial step (m)
x2 = 0:dx2:L;
t2 = 15;    % Initial time step
Fo2 = alpha * t2 / L^2;

T2 = T_initial * ones(length(x2), 1); % Initial temperatures for analytical method
T2_solution = T2;
steady_state_reached = false;

while ~steady_state_reached && t2 <= 135
    T_previous = T2_solution(:, end);

    if Fo2 < 0.2
        % Exact solution (5 terms of the infinite series)
        T_next = zeros(size(x2));
        for i = 1:length(x2)
            theta_sum = 0;
            for n = 1:length(lambda)
                theta_n = (4 * sin(lambda(n))) / (2 * lambda(n) + sin(2 * lambda(n))) * ...
                        exp(-lambda(n)^2 * Fo2) * cos(lambda(n) * x2(i) / L);
                theta_sum = theta_sum + theta_n;
            end
            T_next(i) = theta_sum * (T_initial - T_inf) + T_inf;
        end
    else
        % One-term solution
        T_next = zeros(size(x2));
        for i = 1:length(x2)
            T_next(i) = (4 * sin(lambda(1))) / (2 * lambda(1) + sin(2 * lambda(1))) * ...
```

```
                        exp(-lambda(1)^2 * Fo2) * cos(lambda(1) * x2(i) / L) * (T_initial - T_inf) + T_inf;
        end
    end

    T2_solution = [T2_solution, T_next'];
    max_temp_change = max(abs(T_next' - T_previous));

    if max_temp_change < 0.01
        steady_state_reached = true;
    else
        t2 = t2 + 15;
        Fo2 = alpha * t2 / L^2;
    end
end

% Plotting
figure;
hold on;

% Numerical method plots
for i = 1:size(results1, 2)
    plot(x1, results1(:, i), 'DisplayName', ['Numerical, t = ', num2str((i-1)*15), ' s']);
end

% Analytical method plots
for i = 1:size(T2_solution, 2)
    plot(x2, T2_solution(:, i), '--', 'DisplayName', ['Analytical, t = ', num2str((i-1)*15), ' s']);
end

legend('show');
xlabel('Position(m)');
ylabel('Temperature( C )');
title('Comparison_of_Numerical_and_AnalyticalMethods');
grid on;
```